Sequential Task:
Layout Managers

Object-Oriented Programming and GUI

May 18, 2026

1. Implement the constructor of the `L00NullLayoutPanel` class, which demonstrates the use of the absolute positioning `null layout` in the Swing library.

   (a) Set the component layout manager to `null`.
   - Call the method `setLayout(null)`.

   (b) Set the preferred panel size to `500 x 300` pixels.
   - Use `setPreferredSize(new Dimension(500, 300))`.

   (c) Set the panel border as a titled border with a red line.
   - Create a line border using `BorderFactory.createLineBorder(Color.RED, 2)`.
   - Create a titled border using `BorderFactory.createTitledBorder(...)`.
   - Set the title to `"Null Layout - absolute positioning (NOT RECOMMENDED!)"`.

   (d) Set the panel background color to light red.
   - Use the object `new Color(255, 245, 245)`.

   (e) Create the button `btn1` with the text `"Button 1"`.
   - Set its position and size using `setBounds(20, 30, 100, 40)`.

   (f) Create the button `btn2` with the text `"Button 2"`.
   - Set its position and size using `setBounds(150, 80, 120, 35)`.

   (g) Create the button `btn3` with the text `"Button 3"`.
   - Set its position and size using `setBounds(50, 150, 80, 80)`.

   (h) Create the label `label` with the text `"Label at position (300, 50)"`.
   - Set its position and size using `setBounds(300, 50, 180, 25)`.
   - Enable background filling using `setOpaque(true)`.
   - Set the label background color to red using `setBackground(Color.RED)`.

   (i) Create the text field `field` with the default content `"Text field"`.
   - Set its position and size using `setBounds(280, 150, 150, 30)`.

   (j) Create the warning label `warning` with the text `"bad idea"`.
   - Set its position and size using `setBounds(250, 200, 230, 60)`.
   - Enable background filling using `setOpaque(true)`.

- Set the background color to `new Color(255, 200, 200)`.
- Set a red line border using `BorderFactory.createLineBorder(Color.RED)`.

(k) Add all created components to the panel.

- Add in order: `btn1`, `btn2`, `btn3`, `label`, `field`, and `warning`.

2. Run the application and check whether the result looks as follows:



3. Implement the constructor of the `L01FlowLayoutPanel` class, which demonstrates how the `FlowLayout` layout manager works with different component alignment options.

(a) Create the `demoContainer` panel with a `GridLayout`.

- Set the layout to `3` rows and `1` column.
- Set the horizontal and vertical gaps to `5`.

(b) Create the `leftFlow` panel using the `createDemoPanel()` method.

i. Set the panel layout to `FlowLayout` with `FlowLayout.LEFT` alignment.
ii. Set the horizontal and vertical gaps to `5`.
iii. Add a titled border with the text `"FlowLayout.LEFT"`.
iv. Add six buttons in a loop from `1` to `6`.

- For each button, call `createStyledButton("Button " + i, i)`.
- Add the created button to the `leftFlow` panel.

(c) Create the `centerFlow` panel using the `createDemoPanel()` method.

i. Set the panel layout to `FlowLayout` with `FlowLayout.CENTER` alignment.
ii. Set the horizontal and vertical gaps to `10`.
iii. Add a titled border with the text `"FlowLayout.CENTER"`.
iv. Add six buttons in a loop from `1` to `6`.

- For each button, call `createStyledButton("Btn " + i, i + 2)`.
- Add the created button to the `centerFlow` panel.

(d) Create the `rightFlow` panel using the `createDemoPanel()` method.

i. Set the panel layout to `FlowLayout` with `FlowLayout.RIGHT` alignment.

ii. Set the horizontal gap to `15` and the vertical gap to `5`.

iii. Add a titled border with the text `"FlowLayout.RIGHT"`.

iv. Add six buttons in a loop from `1` to `6`.

- For each button, call `createStyledButton("B" + i, i + 4)`.
- Add the created button to the `rightFlow` panel.

(e) Add the demonstration panels to the `demoContainer`.

- Add in order: `leftFlow`, `centerFlow`, and `rightFlow`.

(f) Add the `demoContainer` to the current panel in the `BorderLayout.CENTER` area.

4. Run the application and check whether the result looks as follows:



5. Implement the constructor of the `L02BorderLayoutPanel` class, which demonstrates how the `BorderLayout` layout manager works.

(a) Create the `demo` panel using the `createDemoPanel()` method.

(b) Set the layout of the `demo` panel to `BorderLayout`.

- Set the horizontal and vertical gaps to `10`.

(c) Create the `north` button by calling `createStyledButton("NORTH - head", 0)`.

- Set the preferred button height to `50` pixels using `setPreferredSize(new Dimension(0, 50))`.

(d) Create the `south` button by calling `createStyledButton("SOUTH - bottom", 1)`.

- Set the preferred button height to `50` pixels.

(e) Create the `east` button by calling `createStyledButton(...)`.

- Pass HTML text displaying the letters of the word `EAST` vertically.

- Set the preferred button width to `80` pixels using `setPreferredSize(new Dimension(80, 0))`.

(f) Create the `west` button by calling `createStyledButton(...)`.

- Pass HTML text displaying the letters of the word `WEST` vertically.
- Set the preferred button width to `80` pixels.

(g) Create the `center` button by calling `createStyledButton(...)`.

- Pass HTML text containing the text `CENTER`.

(h) Add the `north` button to the `BorderLayout.NORTH` area.

(i) Add the `south` button to the `BorderLayout.SOUTH` area.

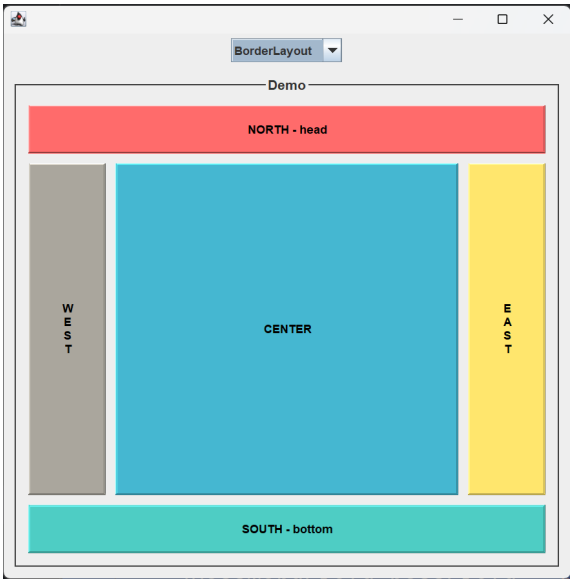
(j) Add the `east` button to the `BorderLayout.EAST` area.

(k) Add the `west` button to the `BorderLayout.WEST` area.

(l) Add the `center` button to the `BorderLayout.CENTER` area.

(m) Add the `demo` panel to the current component in the `BorderLayout.CENTER` area.

6. Run the application and check whether the result looks as follows:



7. Implement the constructor of the `L03GridLayoutPanel` class, which demonstrates the use of the `GridLayout` layout manager in two different applications.

(a) Create the `demoContainer` panel with a `GridLayout`.

- Set the layout to `1` row and `2` columns.
- Set the horizontal and vertical gaps to `10`.

(b) Create the `calc` panel using the `createDemoPanel()` method.

i. Set the layout of the `calc` panel to `BorderLayout` with gaps of `5`.

ii. Add a titled border with the text `"Calc - GridLayout(4,4)"`.

iii. Create the text field `display` with the initial value `"0"`.

iv. Set the text field font to:
- family `"Arial"`,
- style `Font.BOLD`,
- size `24`.

v. Set the text alignment in the field to `JTextField.RIGHT`.

vi. Add the `display` field to the `BorderLayout.NORTH` area.

vii. Create the `buttons` panel with the layout `GridLayout(4, 4, 3, 3)`.

viii. Create the `calcButtons` array containing the calculator button labels:
- `"7","8","9","/","4","5","6","*","1","2","3","-","0",".","=","+"`

ix. Iterate through all elements of the `calcButtons` array using a loop.

x. For each element:
- create a button by calling `createStyledButton(calcButtons[i], i)`,
- add the button to the `buttons` panel.

xi. Add the `buttons` panel to the `BorderLayout.CENTER` area.

(c) Create the `colors` panel using the `createDemoPanel()` method.

i. Set the layout of the `colors` panel to `GridLayout(3, 3, 5, 5)`.

ii. Add a titled border with the text `"Palette - GridLayout(3,3)"`.

iii. Create a loop that runs `9` times.

iv. In each iteration:
- create a new `JPanel` named `colorPanel`,
- set the panel background color to `this.colors[i]`,
- set a black line border using `BorderFactory.createLineBorder(Color.BLACK)`,
- add the panel to the `colors` container.

(d) Add the `calc` panel to the `demoContainer`.

(e) Add the `colors` panel to the `demoContainer`.

(f) Add the `demoContainer` to the current component in the `BorderLayout.CENTER` area.

8. Run the application and check whether the result looks as follows:



9. 🏁 Milestone 1: present the result of your work to the instructor

10. Implement the constructor of the `L04CardLayoutPanel` class, which demonstrates the use of the `CardLayout` layout manager to switch between cards.

    (a) Create a `CardLayout` object and assign it to the `cardLayout` field.

    (b) Create the `cardPanel` panel with the `cardLayout`.

    (c) Set the `cardPanel` border as a titled border with the text `"Cards"`.

    (d) Create four cards in a loop from `1` to `4`.

        i. Create the `card` panel with a `BorderLayout`.

        ii. Set the card background color to the value `colors[i]`.

        iii. Create the `label` label with the text `"Card " + i` and `JLabel.CENTER` alignment.

        iv. Set the label font to `Arial`, style `Font.BOLD`, and size `48`.

        v. Add the label to the center of the card in the `BorderLayout.CENTER` area.

        vi. Add the card to the `cardPanel` under the name `"Card" + i`.

    (e) Create the navigation panel `navPanel` with a `FlowLayout`.

        - Set the alignment to `FlowLayout.CENTER`.
        - Set the horizontal and vertical gaps to `10`.

    (f) Create the navigation buttons `firstBtn`, `prevBtn`, `nextBtn`, and `lastBtn`.

        - Set their texts respectively to `"« first"`, `"< previous"`, `"next >"`, and `"last »"`.

    (g) Add event handling for the navigation buttons.

        - For `firstBtn`, call `cardLayout.first(cardPanel)`.
        - For `prevBtn`, call `cardLayout.previous(cardPanel)`.
        - For `nextBtn`, call `cardLayout.next(cardPanel)`.
        - For `lastBtn`, call `cardLayout.last(cardPanel)`.

(h) Add the buttons `firstBtn`, `prevBtn`, `nextBtn`, and `lastBtn` to the `navPanel`.

(i) Create a drop-down list `cardSelector` of type `JComboBox<String>`.

- Fill it with the values `"Card1"`, `"Card2"`, `"Card3"`, and `"Card4"`.

(j) Add event handling for selecting an item in `cardSelector`.

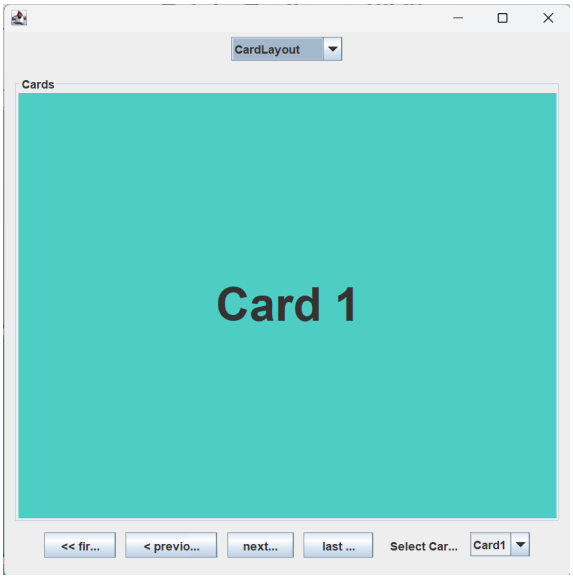
- Retrieve the currently selected card name using `cardSelector.getSelectedItem()`.
- Cast the result to `String`.
- Call `cardLayout.show(cardPanel, ...)` to display the selected card.

(k) Add a label with the text `" Select Card: "` and the `cardSelector` component to the `navPanel`.

(l) Add the `cardPanel` to the current component in the `BorderLayout.CENTER` area.

(m) Add the `navPanel` to the current component in the `BorderLayout.SOUTH` area.

11. Run the application and check whether the result looks as follows:



12. Implement the constructor of the `L05GridBagLayoutPanel` class, which demonstrates the use of the `GridBagLayout` layout manager to build a form.

(a) Create the `demo` panel using the `createDemoPanel()` method.

(b) Set the layout of the `demo` panel to `GridBagLayout`.

(c) Add a titled border with the text `"GridBagLayout form"`.

(d) Create a `GridBagConstraints` object named `gbc`.

(e) Set component margins by assigning the object `new Insets(5, 5, 5, 5)` to `gbc.insets`.

(f) Add the form title.

i. Set `gbc.gridx` to `0` and `gbc.gridy` to `0`.

ii. Set `gbc.gridwidth` to `3`.

iii. Set `gbc.fill` to `GridBagConstraints.HORIZONTAL`.

iv. Create the label `title` with the text `"Registration form:"` and `JLabel.CENTER` alignment.

v. Set the label font to `Arial`, style `Font.BOLD`, and size `16`.

vi. Enable background filling for the label using `setOpaque(true)`.

vii. Set the label background color to `colors[0]`.

viii. Set the internal label margin using `BorderFactory.createEmptyBorder(10, 10, 10, 10)`.

ix. Add the `title` label to the `demo` panel using the `gbc` constraints.

(g) Add the form field for the name.

i. Set `gbc.gridwidth` to `1`.

ii. Set `gbc.gridy` to `1`.

iii. Set `gbc.gridx` to `0`.

iv. Set `gbc.fill` to `GridBagConstraints.NONE`.

v. Set `gbc.anchor` to `GridBagConstraints.EAST`.

vi. Add the label `"Name:"` to the `demo` panel.

vii. Set `gbc.gridx` to `1`.

viii. Set `gbc.gridwidth` to `2`.

ix. Set `gbc.fill` to `GridBagConstraints.HORIZONTAL`.

x. Set `gbc.weightx` to `1`.

xi. Add the text field `new JTextField(20)` to the `demo` panel.

(h) Add the form field for the email address.

i. Restore `gbc.gridwidth` to `1` and `gbc.weightx` to `0`.

ii. Set `gbc.gridy` to `2`.

iii. Set `gbc.gridx` to `0`.

iv. Set `gbc.fill` to `GridBagConstraints.NONE`.

v. Add the label `"Email:"` to the `demo` panel.

vi. Set `gbc.gridx` to `1`.

vii. Set `gbc.gridwidth` to `2`.

viii. Set `gbc.fill` to `GridBagConstraints.HORIZONTAL`.

ix. Set `gbc.weightx` to `1`.

x. Add the text field `new JTextField(20)` to the `demo` panel.

(i) Add the form field for the message.

i. Restore `gbc.gridwidth` to `1` and `gbc.weightx` to `0`.

ii. Set `gbc.gridy` to `3`.

iii. Set `gbc.gridx` to `0`.

iv. Set `gbc.fill` to `GridBagConstraints.NONE`.

v. Set `gbc.anchor` to `GridBagConstraints.NORTHEAST`.

vi. Add the label `"Message"` to the `demo` panel.

vii. Set `gbc.gridx` to `1`.

viii. Set `gbc.gridwidth` to `2`.

ix. Set `gbc.fill` to `GridBagConstraints.BOTH`.

x. Set `gbc.weightx` to `1` and `gbc.weighty` to `1`.

xi. Create a multiline text field `new JTextArea(5, 20)`.

xii. Place it in a `JScrollPane`.

xiii. Add the scrollable text area to the `demo` panel.

(j) Add the form buttons.

i. Set `gbc.weighty` to `0`.

ii. Set `gbc.gridy` to `4`.

iii. Set `gbc.gridx` to `1`.

iv. Set `gbc.gridwidth` to `1`.

v. Set `gbc.fill` to `GridBagConstraints.NONE`.

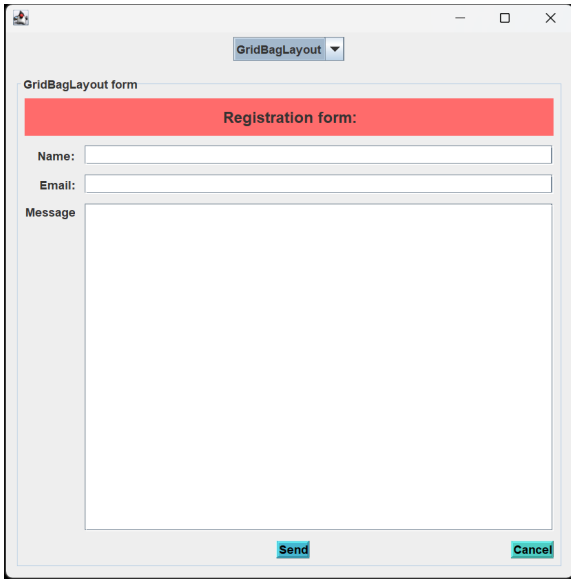
vi. Add the `"Send"` button created by `createStyledButton("Send", 4)`.

vii. Set `gbc.gridx` to `2`.

viii. Add the `"Cancel"` button created by `createStyledButton("Cancel", 1)`.

(k) Add the `demo` panel to the current component in the `BorderLayout.CENTER` area.

13. Run the application and check whether the result looks as follows:



14. Implement the constructor of the `L06SpringLayoutPanel` class, which demonstrates the use of the `SpringLayout` layout manager to manually define position relationships between form components.

(a) Create the `demo` panel using the `createDemoPanel()` method.

(b) Create a `SpringLayout` object named `springLayout`.

(c) Set the layout of the `demo` panel to `springLayout`.

(d) Set the preferred size of the `demo` panel to `400 x 200` pixels.

(e) Add a titled border with the text `"SpringLayout form"`.

(f) Create the form components.

i. Create the label `nameLabel` with the text `"User name:"`.

ii. Create the text field `nameField` with a width of `15` columns.

iii. Create the label `passLabel` with the text `"Password:"`.

iv. Create the password field `passField` with a width of `15` columns.

v. Create the `loginBtn` button with the text `"Login"` by calling `createStyledButton("Login", 4)`.

vi. Create the `cancelBtn` button with the text `"Cancel"` by calling `createStyledButton("Cancel", 0)`.

(g) Add all created components to the `demo` panel.

- Add in order: `nameLabel`, `nameField`, `passLabel`, `passField`, `loginBtn`, and `cancelBtn`.

(h) Set position constraints for the `nameLabel` label.

i. Set the left edge of `nameLabel` `10` pixels from the left edge of the `demo` panel.

ii. Set the top edge of `nameLabel` `10` pixels from the top edge of the `demo` panel.

(i) Set position constraints for the `nameField` field.

i. Set the left edge of `nameField` `10` pixels from the right edge of `nameLabel`.

ii. Set the top edge of `nameField` `10` pixels from the top edge of the `demo` panel.

iii. Set the right edge of `nameField` `10` pixels from the right edge of the `demo` panel.

(j) Set position constraints for the `passLabel` label.

i. Set the left edge of `passLabel` `10` pixels from the left edge of the `demo` panel.

ii. Set the top edge of `passLabel` `15` pixels below the bottom edge of `nameLabel`.

(k) Set position constraints for the `passField` field.

i. Align the left edge of `passField` with the left edge of `nameField`.

ii. Set the top edge of `passField` `10` pixels below the bottom edge of `nameField`.

iii. Set the right edge of `passField` `10` pixels from the right edge of the `demo` panel.

(l) Set position constraints for the `cancelBtn` button.

i. Set the right edge of `cancelBtn` `10` pixels from the right edge of the `demo` panel.

ii. Set the bottom edge of `cancelBtn` `10` pixels from the bottom edge of the `demo` panel.

(m) Set position constraints for the `loginBtn` button.

i. Set the right edge of `loginBtn` `10` pixels from the left edge of `cancelBtn`.

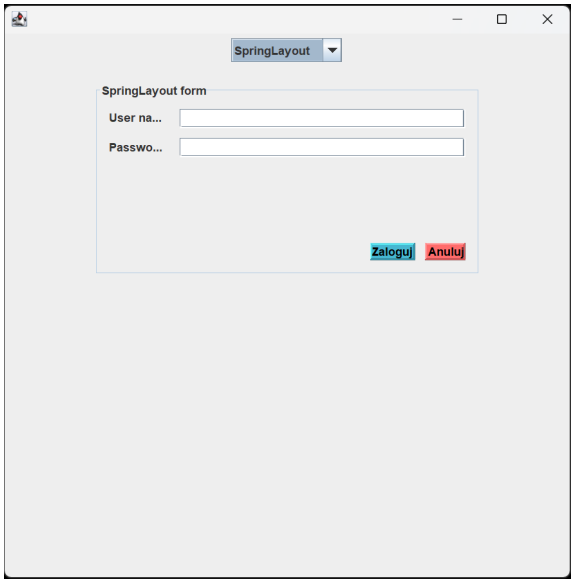
ii. Set the bottom edge of `loginBtn` `10` pixels from the bottom edge of the `demo` panel.

(n) Create the wrapper panel `wrapper` with a `FlowLayout`.

(o) Add the `demo` panel to the `wrapper` panel.

(p) Add the `wrapper` panel to the current component in the `BorderLayout.CENTER` area.

15. Run the application and check whether the result looks as follows:



16. Implement the constructor of the `L07GroupLayoutPanel` class, which demonstrates the use of the `GroupLayout` layout manager to build a form with aligned labels, text fields, and buttons.

    (a) Create the `demo` panel using the `createDemoPanel()` method.

    (b) Add a titled border with the text `"GroupLayout form"` to the `demo` panel.

    (c) Create a `GroupLayout` object named `layout`, passing the `demo` panel to the constructor.

    (d) Set the layout of the `demo` panel to the created `layout` object.

    (e) Enable automatic gaps between components by calling `layout.setAutoCreateGaps(true)`.

    (f) Enable automatic gaps between components and container edges by calling `layout.setAutoCreateCo`

    (g) Create the form labels.

        i. Create the label `l1` with the text `"Name:"`.

        ii. Create the label `l2` with the text `"Surname:"`.

        iii. Create the label `l3` with the text `"Email:"`.

        iv. Create the label `l4` with the text `"Phone:"`.

    (h) Create the form text fields.

        i. Create the text field `f1` with a width of `20` columns.

        ii. Create the text field `f2` with a width of `20` columns.

        iii. Create the text field `f3` with a width of `20` columns.

        iv. Create the text field `f4` with a width of `20` columns.

    (i) Create the form buttons.

i. Create the `submit` button with the text `"Save"` by calling `createStyledButton("Save", 4)`.

ii. Create the `clear` button with the text `"Load"` by calling `createStyledButton("Load", 1)`.

(j) Define the horizontal layout group by calling `layout.setHorizontalGroup(...)`.

i. Create the main sequential group, which places the label column next to the column of text fields and buttons.

ii. Add the first parallel group with `GroupLayout.Alignment.TRAILING` alignment.

- Place the components `l1`, `l2`, `l3`, and `l4` inside it.
- Ensure that the labels are aligned to the right side of the column.

iii. Add the second parallel group with `GroupLayout.Alignment.LEADING` alignment.

- Place the text fields `f1`, `f2`, `f3`, and `f4` inside it.
- Add a nested sequential group containing the buttons `submit` and `clear`.

(k) Define the vertical layout group by calling `layout.setVerticalGroup(...)`.

i. Create the main sequential group, which arranges consecutive form rows one below another.

ii. Add a parallel group with `GroupLayout.Alignment.BASELINE` alignment for the first row.

- Place the label `l1` and the text field `f1` inside it.

iii. Add a parallel group with `GroupLayout.Alignment.BASELINE` alignment for the second row.

- Place the label `l2` and the text field `f2` inside it.

iv. Add a parallel group with `GroupLayout.Alignment.BASELINE` alignment for the third row.

- Place the label `l3` and the text field `f3` inside it.

v. Add a parallel group with `GroupLayout.Alignment.BASELINE` alignment for the fourth row.

- Place the label `l4` and the text field `f4` inside it.

vi. Add a parallel group with `GroupLayout.Alignment.BASELINE` alignment for the button row.

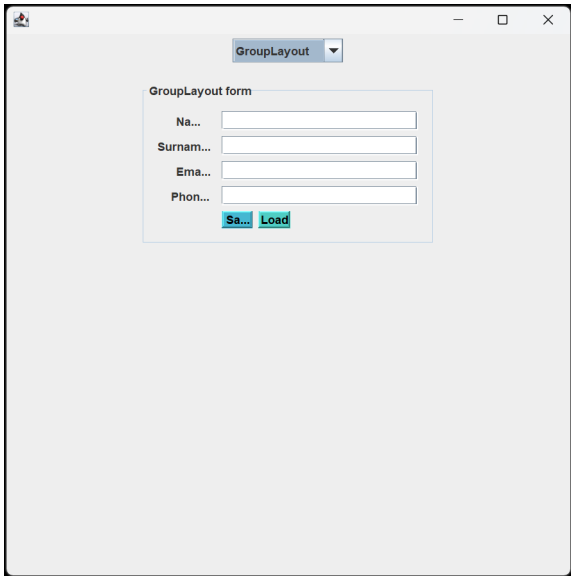
- Place the buttons `submit` and `clear` inside it.

(l) Create the wrapper panel `wrapper` with a `FlowLayout`.

(m) Add the `demo` panel to the `wrapper` panel.

(n) Add the `wrapper` panel to the current component in the `BorderLayout.CENTER` area.

17. Run the application and check whether the result looks as follows:



18. ★ Milestone 2: present the result of your work to the instructor